

Documento Requisiti — Ruffino Ops (PRD)

Stato: Documento vivente, riallineato allo stato corrente dell'applicazione. **Versione:** 3.0 — Aggiornamento esteso (post-audit di sicurezza, soft-archive, dual-address, detrazione tipizzata, classifica venditori, doc-gate con bypass). **Riferimento implementativo:** repository `infissi-ops-app`. Il presente PRD descrive il comportamento atteso del software così come è implementato; ogni divergenza riscontrata nel codice va trattata come bug.

0. Convenzioni del documento

- **MUST / DEVE** — requisito obbligatorio.
 - **SHOULD / DOVREBBE** — requisito fortemente consigliato.
 - **MAY / PUÒ** — opzione facoltativa.
 - Gli identificatori `in_questo_stile` sono valori interni (enum, chiavi di database). Le etichette UI in italiano sono indicate fra virgolette.
 - I numeri di sezione sono stabili; le sotto-sezioni possono crescere.
-

1. Visione del prodotto

Ruffino Ops è lo strumento operativo centrale di **Ruffino Immobiliare S.R.L.** Collega ufficio, laboratorio di produzione e cantiere, accompagnando ogni cliente dalla prima richiesta fino alla garanzia post-vendita. Non è un semplice database: è un **assistente proattivo** che ricorda le scadenze in base alla priorità delle commesse, blocca i passaggi di stato senza i documenti richiesti, e fornisce viste operative (Board, Calendario, Classifica) pensate per le persone reali che le useranno tutti i giorni.

Pilastrì:

1. **Una commessa = un percorso tracciato.** Stato, documenti, interventi, anomalie e firme convivono in un unico fascicolo.
 2. **Niente dato perso.** Le commesse rifiutate dal cliente vengono **archivate**, non cancellate; in qualsiasi momento si possono ripristinare con stato e file invariati.
 3. **Sicurezza by default.** Autenticazione obbligatoria su ogni endpoint; password hashate; gate documentale lato server.
 4. **Operatività prima dell'eleganza.** Pagine come Calendario e Board mostrano in faccia all'utente le informazioni che gli servono (nome, cognome, indirizzo lavoro, telefono cliccabile).
-

2. Architettura tecnica (sintesi)

- **Frontend.** React 19 + Vite + Wouter (routing) + tRPC v10 (client) + React Query (caching) + Tailwind + shadcn/ui + lucide-react + sonner (toast) + jsPDF/jspdf-autotable per i PDF dei preventivatori.

- **Backend.** Node + Express + tRPC v10. Persistenza in `kv_store` (Postgres JSONB) tramite un piccolo layer `persistedStore` con save debounce, retry su errori transienti, recovery in background.
 - **Autenticazione.** Locale via email/password con JWT firmato (jose, HS256, TTL 7 giorni) + cookie `httpOnly`. Sessione server-side cacheata in memoria con eviction periodica.
 - **Sicurezza.** Tutti gli endpoint business sono `protectedProcedure` (utente loggato obbligatorio); le mutazioni su `utenti` sono `adminProcedure` (ruolo direzione). Header `X-Content-Type-Options`, `X-Frame-Options=SAMEORIGIN`, `Referrer-Policy`, HSTS in produzione. Upload con allowlist `mimeType` + validazione reale del payload base64.
 - **Storage file.** Documenti caricati come base64 nel KV store, separati per dominio (`preventivi_documento`, `ticket_allegati`). Cap per-file 10 MB.
-

3. Autenticazione e sicurezza

3.1 Login

- Schermata `LoginPage` con email + password.
- Login solo per utenti `attivo === true`.
- Confronto password tramite `verifyPassword` (scrypt) — accetta anche password legacy in chiaro durante una migrazione trasparente.
- **Rate limit per email:** dopo 5 tentativi falliti in 15 minuti l'account è bloccato fino allo scadere della finestra. Login riuscito → reset del contatore.
- Messaggio di errore generico ("Email o password non validi") per non rivelare se l'utente esiste.

3.2 Sessione

- JWT firmato HS256, claim: `sub` (id utente), `email`, `name`, `role`, `ruolo`, `ruoli`. Esp. 7 giorni.
- Cookie `httpOnly`, `secure` su HTTPS, `sameSite none` su https / `lax` su http locale.
- Cache server-side `Map<token, { user, expMs }>` con eviction lazy e sweep orario (`setInterval(...).unref()`).
- **Logout** cancella sia il cookie sia l'entry della cache server.

3.3 Hashing password

- Algoritmo: **scrypt** (modulo `crypto` di Node, nessuna dipendenza esterna).
- Formato di archiviazione: `scrypt$<saltHex>$<hashHex>` (salt 16 byte, key 64 byte).
- `verifyPassword` è constant-time tramite `timingSafeEqual`.
- Le password create/aggiornate dalla UI sono sempre hashate.
- Le password legacy in chiaro residue nel DB vengono **automaticamente upgradeate a hash al primo boot successivo** all'aggiornamento (vedi `utenti.onLoad`).

3.4 Segreto JWT

- In **produzione** la variabile d'ambiente `JWT_SECRET` è OBBLIGATORIA: in sua assenza il server fa throw all'avvio.
- In dev/test, in mancanza di `JWT_SECRET`, viene usato un fallback hard-coded; **non valido per produzione**.

3.5 Header di sicurezza

Su ogni risposta HTTP:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: SAMEORIGIN`
- `Referrer-Policy: strict-origin-when-cross-origin`
- `X-DNS-Prefetch-Control: off`
- `X-Permitted-Cross-Domain-Policies: none`
- In produzione: `Strict-Transport-Security: max-age=15552000; includeSubDomains`
- CSP **non** impostata di default (richiede tuning con Vite + Maps proxy + blob preview) — tracciata come follow-up.

3.6 Esposizione API

- Endpoint tRPC montati su `/api/trpc`.
- Tutti i router business utilizzano `protectedProcedure`. Le sole eccezioni pubbliche sono: `auth.me`, `auth.login`, `auth.logout`, `system.health`.
- `utenti.create`, `utenti.update`, `utenti.delete` richiedono `adminProcedure` (= ruolo `direzione`).

3.7 Operatività residua a carico del titolare (non risolvibile via codice)

- Rotazione manuale delle password committate in git history prima della migrazione a hash.
- Pulizia dello storico Git da quelle password (`BFG / git filter-repo` + `force-push`).

4. Ruoli, permessi e gating

4.1 Set di ruoli

- `direzione` — ammessi accesso completo + gestione utenti + sezioni gated (Squadre, Garanzie, Fornitori, Produzione, Utenti, Integrazioni avanzate).
- `amministrazione` — focus su fatturazione, finiture, saldo.
- `commerciale` — venditori; partecipano alla **Classifica**.
- `tecnico_rilievi` — rilievi e misure.
- `squadra_posa` — esecuzione in cantiere.
- `post_vendita` — ticket, reclami, rifacimenti.
- `ordini` — ordini fornitori.

Ogni utente ha `ruoli: string[]` (1–3 valori). Il campo legacy `ruolo` continua a contenere il ruolo primario per retro-compatibilità.

4.2 Mapping `role` legacy

- `role = "admin"` quando in `ruoli` è presente `direzione`. Altrimenti `user`.

4.3 Gating

- **Lato server.** `protectedProcedure` su tutto il business; `adminProcedure` su mutazioni `utenti` e `system.notifyOwner`.
 - **Lato client.** Componente `RequireDirezione` su rotte `Garanzie`, `Squadre`, `Fornitori`, `Produzione`, `Utenti`. Le voci di sidebar corrispondenti sono filtrate con il flag `direzioneOnly`.
-

5. Anagrafica Cliente

5.1 Tipi cliente

`privato` | `azienda` | `condominio` | `ente_pubblico`. Icone in UI: `User`, `Building2`, `Home`, `Landmark`.

5.2 Campi del Cliente

Anagrafica:

- **Nome** (obbligatorio).
- **Cognome** (obbligatorio).
- **Tipo** (vedi 5.1; default `privato`).
- **Codice fiscale, partita IVA**.

Doppio indirizzo:

- **Indirizzo di residenza** (`indirizzo`, `citta`, `cap`) — usato dall'amministrazione per la fatturazione.
- **Indirizzo dei lavori** (`indirizzoLavoro`, `cittaLavoro`, `capLavoro`) — usato dalle commesse per cantiere, calendario, mappe.
- In fase di creazione/modifica il form propone uno **switch "stesso della residenza"** che copia automaticamente i campi residenza → lavoro al salvataggio.
- La scheda cliente espone entrambi gli indirizzi con badge distintivo "Residenza" / "Lavoro".

Contatti: **telefono, email**.

Dati fiscali e amministrativi:

- **Detrazione** (switch). Quando attivata RICHIEDE `tipoDetrazione`.
- **Tipo detrazione:** `ecobonus` | `ristrutturazione` (obbligatorio se `detrazione === true`).
- **Finanziamento** (switch).
- **Pratica edilizia:** `nessuna` | `cil` | `cila` | `scia`.

Referenti (lista multipla):

- Campi per referente: `nome`, `ruolo` (`cliente_finale` | `architetto` | `direttore_lavori` | `amministratore` | `altro`), `telefono`, `email`.

Operativi:

- **Note libere**.

- **Assegnato a** (utente). Default: utente corrente; eredita su nuove commesse linkate al cliente.

5.3 Comportamenti del Cliente

- Lista clienti con ricerca testuale, filtro per tipo, filtro "solo le mie".
- Scheda cliente con tabs: **Commesse**, **Interventi**, **Ticket**, **Garanzie** — ognuno ha un pulsante **+** per creare l'entità collegata.
- **Cascade su update**. Modificando nome/cognome o un campo di contatto/indirizzo del cliente, il server propaga il valore a tutte le commesse linkate (solo dove il valore della commessa coincideva con il valore precedente del cliente, così le override manuali non vengono sovrascritte).
- Eliminazione cliente disponibile (con conferma) ma **SCONSIGLIATA** in presenza di commesse linkate.

6. Commesse

6.1 Identificazione

- **Codice automatico** `COM-{ANNO}-{N}` con N progressivo a 3 cifre (es. `COM-2026-001`).
- Lo ZeroPadding è coerente per anno; al cambio anno il contatore può ripartire da 1.

6.2 Campi

- **clientId** (FK), **cliente** (display name denormalizzato, sincronizzato da `cliente.update`).
- **indirizzo**, **citta**, **telefono**, **email** — inizialmente derivati dall' `indirizzoLavoro` del cliente; sovrascrivibili a livello di commessa.
- **stato** — vedi 7.1.
- **priorita**: `bassa` | `media` | `alta` | `urgente`.
- **squadraId** (FK opzionale).
- **dataApertura** (auto).
- **consegnaIndicativa**: enum `"30"` | `"60"` | `"90"` (giorni dal preventivo).
- **dataConsegnaIndicativa**: data libera ISO. **Mutuamente esclusiva** con `consegnaIndicativa`: salvando una delle due l'altra viene azzerata.
- **dataConsegnaConfermata**: data definitiva impostata al passaggio in `produzione`.
- **dataChiusura**: impostata automaticamente al passaggio in `archiviata` (chiusura del flusso).
- **note** libere.
- **prodotti**: array di prodotti desiderati (nome, tipologia, quantità, dimensioni, note). NON viene incluso nella risposta di `commesse.list` per alleggerire i payload; viene ritornato da `commesse.byId`.
- **assegnatoA** (FK utente). Modificabile dalla scheda commessa.
- **createdBy** (FK utente).
- **createdAt**, **updatedAt**.
- **archivedAt**: timestamp ISO; `null` finché la commessa non è in soft-archive (vedi §24).

6.3 Consegna indicativa — selezione

La UI offre quattro opzioni nel select **Consegna indicativa**:

1. `+30 giorni`

2. `+60` giorni (*default*)
3. `+90` giorni
4. **Data da calendario...** → mostra un date picker, popola `dataConsegnaIndicativa` e svuota `consegnaIndicativa`.

L'header della commessa mostra in ordine di priorità:

1. `dataConsegnaConfermata` (etichetta "Data consegna prevista").
2. `dataConsegnaIndicativa` (etichetta "Consegna indicativa", formato `gg/mm/aaaa`).
3. `consegnaIndicativa` (etichetta "Consegna indicativa: +N giorni").

6.4 Trigger Produzione

Quando una commessa entra nello stato `produzione` :

1. Sulla scheda compare un banner ambra "Commessa in produzione" con il pulsante "Aggiorna data consegna".
2. Al click si apre un dialog con date picker per la **Data di Consegna Prevista** definitiva.
3. Il salvataggio popola `dataConsegnaConfermata` (visibile in header, card Kanban, lista in Dashboard).
4. Finché `dataConsegnaConfermata` è vuota, la card del Kanban resta evidenziata con anello ambra e contatore "Consegne da confermare" nel KPI bar.

6.5 Modifica

- Dialog "**Modifica commessa**" consente di modificare contemporaneamente:
 - Anagrafica cliente collegata (nome, cognome, CF, P.IVA, CAP, telefono, email, indirizzo, città) — sincronizzata via `clienti.update` con cascade.
 - Priorità.
 - **Utente assegnato** (SearchSelect sugli utenti, opzione "Non assegnata").
 - Consegna indicativa (preset o data libera).
 - Note.
- Nell'header sono visibili pill: indirizzo, telefono, email, **Assegnata a: Nome** (lookup utente).

6.6 Eliminazione vs Archiviazione

- **Archivia** — operazione consigliata se il cliente non procede. Non distrugge nulla (vedi §24).
- **Elimina** — operazione distruttiva. Conferma esplicita. Dovrebbe essere usata solo per errori di inserimento.

6.7 Lista commesse

- Filtri: search testuale (codice, cliente, città), stato, clienteld, assegnatoA, scope `archived = exclude | only | all` (default `exclude`).
 - Ordinata per `createdAt` desc.
 - Risposta **non include** `prodotti` (ottimizzazione bandwidth/render).
-

7. State machine delle commesse

7.1 Stati

Ordine canonico (STATI_COMMESSA):

1. preventivo
2. misure_esecutive
3. aggiornamento_contratto
4. fatture_pagamento
5. da_ordinare
6. produzione
7. ordini_ultimazione (*richiesta secondo acconto*)
8. attesa_posa
9. finiture_saldo
10. interventi_regolazioni
11. archiviata (*chiusura del flusso — distinta dal soft-archive di §24*)

7.2 Transizioni valide

- Transizioni **in avanti** consentite di **una posizione** (es. da_ordinare → produzione).
- Transizioni **all'indietro** consentite di **una posizione**.
- Salti multipli **vietati** lato server (validateTransizione).
- Stato archiviata raggiungibile solo dal predecessore interventi_regolazioni.

7.3 Soft-archive (orthogonal)

- archivedAt è ortogonale a stato . Una commessa può essere soft-archiviata in qualsiasi stato (vedi §24).
- Il soft-archive **non** modifica lo stato corrente.

7.4 Avanzamento via UI

- **Scheda commessa**: pulsante "Avanza" che propone lo stato successivo. Disabilitato se manca un documento richiesto (vedi §9), eccetto bypass con "Procedi comunque".
- **Board Kanban**: ogni card ha frecce **Indietro** / **Avanza**. Le frecce attraversano sempre la stessa state machine (controlli server identici).

7.5 Stati e gate documentale (vedi anche §9)

- preventivo → necessita preventivo o contratto .
- misure_esecutive → misure .
- aggiornamento_contratto → contratto .
- fatture_pagamento → fattura .
- da_ordinare → ordine o conferma_ordine .
- produzione → nessun documento richiesto (gated da dataConsegnaConfermata).
- ordini_ultimazione → saldo o fattura .

- `attesa_posa` → `ddt_consegna`.
 - `finiture_saldo` → `ddt_posa`.
 - `interventi_regolazioni` → `ddt_finale`.
 - `archiviata` → nessun documento richiesto.
-

8. Documenti commessa (Preventivi/Contratti)

8.1 Tipi documento

`preventivo`, `contratto`, `misure`, `fattura`, `ordine`, `conferma_ordine`, `ddt_consegna`, `ddt_posa`, `ddt_finale`, `saldo`, `foto`, `altro`.

8.2 Storage e schema

- Persistito in `preventivi_documenti` (kv_store JSONB).
- Per ogni documento: `id`, `commessaId`, `nome`, `tipo`, `mimeType`, `size`, `dataBase64`, `note`, `statoAtUpload`, `createdBy`, `createdAt`.
- Lista `byCommessa` strippa `dataBase64` e restituisce un flag `hasData`. Solo `byId` ritorna il contenuto.

8.3 Upload — controlli

- **MimeType allowlist** (stored XSS hardening):
 - `application/pdf`
 - `image/jpeg`, `image/png`, `image/gif`, `image/webp`, `image/heic`, `image/heif`
 - `application/msword`, `application/vnd.openxmlformats-officedocument.wordprocessingml.document`
 - `application/vnd.ms-excel`, `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`
 - **Esclusi esplicitamente** `text/html` e `image/svg+xml`.
- **Dimensione**: validata sul payload reale (lunghezza base64 decodificata, NON il campo `size` lato client). Cap 10 MB.
- Il `size` archiviato è quello calcolato dal server.

8.4 Auto-rename

- Se `keepNome === false`, il file in upload viene rinominato secondo `renameForStato({stato, cliente, originalName})` (es. "Misure esecutive Mario Rossi.pdf").
- Se `keepNome === true` (usato dai preventivatori), il nome viene preservato e solo deduplicato.
- Disambiguazione automatica: se il nome esiste già per la stessa commessa, viene appeso (2), (3), ecc.

8.5 Anteprima e download

- Anteprima inline in `<iframe>` per PDF (URL `blob:` derivato dal base64) o in `<img>` con zoom/rotate per immagini.

- Download via `<a download>`.
- Invio email via `mailto:` con corpo precompilato (no upload server-side dell'allegato).

8.6 Eliminazione

- Soft delete NON previsto. La cancellazione è definitiva.

8.7 Allegati ticket

- Pattern identico (router `ticketAllegati`), stesso schema base64, stessa allowlist mime, stesso size check, cap 10 MB.
- Cancellando un ticket si cancellano in cascata i suoi allegati (`deleteAllegatiByTicket`).

9. Doc gate (gate documentale)

9.1 Regola

- Una transizione **in avanti** verifica che esista almeno un documento con uno dei tipi richiesti dallo stato CORRENTE (`REQUIRED_DOC_TIPI_PER_STATO`).
- Conta solo se il documento è stato caricato **mentre la commessa era in quello stato** (campo `statoAtUpload`), così un preventivo non può soddisfare un gate diverso.
- Per i documenti legacy senza `statoAtUpload`, fallback permissivo: il tipo è sufficiente.

9.2 Stati daily reminder

Per gli stati `aggiornamento_contratto`, `fatture_pagamento`, `da_ordinare` viene generata anche una notifica giornaliera (vedi §27.3) anche oltre la soglia di priorità.

9.3 Bypass "Procedi comunque"

- **Server.** `commesse.update` accetta un flag `force: boolean`. Se assente, il gate è bloccante; in caso di mancanza documenti il server lancia un errore con prefisso `DOC_GATE_BLOCKED:` e messaggio human-readable ("Non è stato caricato il file Procedere comunque?").
- **Client (scheda commessa).** Il pulsante "Avanza" non è più disabilitato dal gate. Al click, se il gate è violato, mostra un `ConfirmDialog` non-destructive con label "Procedi comunque". In conferma chiama `update({ stato, force: true })`.
- **Client (Kanban).** Onerror della mutation: se `err.message` inizia per `DOC_GATE_BLOCKED:`, lo stesso `ConfirmDialog` viene aperto e su conferma rilancia con `force: true`. Errori non-gate restano nel banner inline.
- **La state machine resta invariata.** `force` bypassa SOLO il gate documentale, non la shape delle transizioni.

9.4 Indicatore UI

Sotto l'header della commessa è presente una card con stato del gate:

- Verde se tutti i documenti richiesti sono presenti.

- Ambra se mancano: lista di tipi richiesti con badge "Caricato"/"Mancante" + bottone "Carica file" che preimposta il tipo nel dialog upload.

10. Aperture (Rilievo)

- Una **apertura** è un singolo serramento all'interno di una commessa.
- Campi: `commessaId`, `codice`, `descrizione`, `piano`, `locale`, `tipologia` (`finestra` | `porta-finestra` | `porta` | `scorrevole` | `fisso` | `altro`), `larghezza`, `altezza`, `profondita`, `materiale`, `colore`, `vetro`, `noteRilievo`, `criticitaAccesso`, `stato`.
- Stati apertura: `da_rilevare` → `rilevata` → `ordinata` → `consegnata` → `in_posa` → `posata` → `verificata`.
- Le aperture sono visibili nella scheda commessa e nella vista **RilievoDetail** dedicata.
- Una commessa **può** avere zero aperture (es. preventivo iniziale generico).

11. Board Kanban (/kanban)

11.1 Layout

Quattro **fasi**, ognuna con N colonne:

Fase	Colonne
Vendita	Preventivo, Misure Esecutive, Aggiornamento Contratto
Ordine & Produzione	Fatture / Pagamento, Da Ordinare, Produzione
Consegna & Posa	Richiesta Secondo Acconto, Attesa Posa
Chiusura	Finiture / Saldo, Interventi / Regolaz.

Lo stato `archiviata` non compare.

11.2 Card commessa

Mostra: codice, badge priorità, cliente, città, indicatore di consegna (data confermata o indicativa), pulsanti **Indietro** / **Avanza** con etichetta dello stato target.

11.3 Filtri

- Search per codice/cliente/città.
- Filtro priorità (Tutte/Urgente/Alta/Media/Bassa).
- Toggle "Nascondi vuote" / "Mostra vuote".
- Chip per fase (Tutte le fasi / per singola fase).
- Tutte le fasi possono essere collassate.

11.4 KPI bar

Quattro cards: **Attive**, **Urgenti**, **Alte**, **Consegne da confermare** (commesse in produzione senza dataConsegnaConfermata).

11.5 Doc gate sul Kanban

Le frecce **Avanza** seguono lo stesso doc gate. Errore `DOC_GATE_BLOCKED:` → ConfirmDialog "Procedi comunque". Errori non-gate → banner inline rosso.

11.6 Esclusioni

- Le commesse soft-archivate non appaiono.
-

12. Calendario / Planning (/planning)

12.1 Viste

- **Giorno** — singola colonna.
- **Settimana** (*default*) — 7 colonne lun–dom; weekend leggermente attenuati.
- **Mese** — griglia 6×7.

12.2 Tipi di intervento

rilievo, posa, assistenza, altro. Colori dedicati per tipo.

12.3 Card intervento (joined info)

Ogni card mostra:

- Ora inizio – ora fine.
- Tipo intervento.
- **Nome + cognome** del cliente (lookup commessa → cliente).
- **Indirizzo** (fallback `intervento.indirizzo` → `commessa.indirizzo` → `cliente.indirizzoLavoro` → `cliente.indirizzo`).
- Note brevi.
- Stato (`pianificato` di default).

12.4 Dialog dettagli appuntamento

In modalità modifica, sopra al form, viene mostrato un blocco di sintesi joined:

- Codice commessa, stato, badge priorità.
- Nome cliente.
- Indirizzo cantiere.
- Telefono (link `tel:`).
- Email (link `mailto:`).

- Squadra assegnata.
- Pulsante "Apri commessa" → naviga alla scheda commessa.

12.5 Auto-fill

Quando si seleziona una commessa in dialog di creazione, l'indirizzo viene auto-popolato dalla commessa (solo se il campo è vuoto, per non sovrascrivere override manuali).

12.6 Drag & drop

- Un intervento può essere trascinato fra giorni nelle viste settimana e mese.
- L'update della data avviene via `interventi.update({ dataPianificata })`.

12.7 Stati intervento

`pianificato`, `in_corso`, `completato`, `sospeso`, `annullato`. L'avvio imposta `dataInizio`; la chiusura imposta `dataFine`. Le entry `annullato` legacy sono **purgate** automaticamente al boot (cleanup nel `persistedStore.onLoad`).

12.8 Link entità

Un intervento può essere collegato a una delle entità: `commessa`, `ticket`, `reclamo`, `rifacimento`. Solo `commessa` è obbligatoria quando il `linkKind === "commessa"`.

13. Ticket post-vendita (/ticket e contenuti di /reclami)

13.1 Modello

- Campi: `commessaId`, `aperturaId?`, `oggetto`, `descrizione`, `categoria`, `priorita`, `stato`, `assegnatoA?`, `dataRisoluzione?`, `esitoIntervento?`, `apertoBy?`.
- Categorie: `difetto_prodotto`, `difetto_posa`, `regolazione`, `sostituzione`, `garanzia`, `altro`.
- Priorità: come per le commesse.

13.2 State machine ticket

`aperto` → `assegnato` → `in_lavorazione` → `risolto` → `chiuso`. Disponibile **rollback** di una posizione (clear `dataRisoluzione` se si esce da `risolto/chiuso`). Da `aperto` il rollback è rifiutato.

13.3 Allegati ticket

Vedi §8.7.

14. Reclami e Rifacimenti (/reclami)

Pagina unificata che gestisce due entità correlate ma distinte.

14.1 Reclamo

- Campi: `commessaId`, `clienteNome`, `descrizione`, `responsabile?`, `stato`, `dataApertura`, `dataRisoluzione?`, `soluzione?`.
- Stati: `aperto`, `in_gestione`, `risolto`, `chiuso`.

14.2 Rifacimento

- Campi: `commessaId`, `clienteNome`, `descrizione`, `elemento`, `fornitoreCoinvolto?`, `ordineRifacimentoId?`, `costoStimato?`, `responsabilita (interna|esterna)`, `responsabile?`, `stato`, `dataApertura`, `dataChiusura?`.
- Stati: `aperto`, `in_gestione`, `in_produzione`, `completato`, `chiuso`.

15. Verbali (/verbale/:interventoId + VerbaleChiusura)

- Tipo: `chiusura_lavori` | `sopralluogo` | `consegna` (default `chiusura_lavori`).
- Campi: `interventoId`, `commessaId`, `tipo`, `data`, `noteCliente?`, `noteTecnico?`, `firmaClienteData?`, `firmaTecnicoData?`, `firmaCliente`, `firmaTecnico`, `apertureCompletate`, `apertureTotali`, `anomalieRiscontrate`, `stato` (`bozza`|`firmato`).
- Lo stato passa a `firmato` quando entrambe le firme sono presenti.

16. Anomalie

- Campi: `commessaId`, `aperturaId?`, `interventoId?`, `categoria`, `priorita`, `descrizione`, `risoluzione?`, `stato`, `segnalataBy?`, `risoltaBy?`, `risoltaAt?`.
- Categorie: `materiale_difettoso`, `misura_errata`, `danno_trasporto`, `difetto_posa`, `problema_accessorio`, `non_conformita`, `altro`.
- Priorità: `bassa`, `media`, `alta`, `critica`.
- Stati: `aperta`, `in_gestione`, `risolta`, `chiusa`. Endpoint dedicato `resolve` imposta `stato=risolta`, `risoluzione`, `risoltaAt`.

17. Garanzie (/garanzie , direzione-only)

- Tipi: `prodotto`, `posa`, `accessorio`, `vetro`, `altro`.
- Campi: `commessaId`, `aperturaId?`, `tipo`, `descrizione`, `fornitore?`, `dataInizio`, `durataMesi`, `dataScadenza`, `stato`, `documentoRif?`, `note?`.
- `dataScadenza` calcolata server-side da `dataInizio` + `durataMesi`.

- Stati: attiva, scaduta, sospesa, revocata.
 - Statistiche: totale, attive, in scadenza (entro 90 giorni), scadute.
-

18. Squadre (/squadre , direzione-only)

- Campi: nome, caposquadra?, telefono?, note?, attiva.
 - Le squadre attive appaiono nei dropdown della scheda commessa, dialog intervento, calendario.
 - L'inattivazione (toggle attiva) preserva storico ma rimuove la squadra dai picker.
-

19. Fornitori (/fornitori , direzione-only)

19.1 Anagrafica fornitore

- Campi: ragioneSociale, partitaIva, indirizzo?, citta?, telefono?, email?, categoria, referenteCommerciale?, scontistica?, note?, attivo.
- Categorie: pvc, alluminio, vetro, ferramenta, persiane, blindati, accessori, guarnizioni, altro.

19.2 Ordini fornitore

- Campi: fornitoreId, commessaId, codiceOrdine, stato, dataOrdine, dataConsegnaPrevi-
sta?, dataConsegnaEffettiva?, righe[], noteOrdine?, noteRicevimento?, importoTotale.
- Stati: bozza, inviato, confermato, in_transito, ricevuto_parziale, ricevuto,
contestato.
- Riga ordine: descrizione, codiceArticolo?, quantita, quantitaRicevuta, unitaMisura,
prezzoUnitario?, lotto?, conforme?, noteDifetto?.
- L'update di stato accetta righeAggornate per registrare la ricezione parziale.

19.3 Listini fornitore

- Campi: fornitoreId, nome, versione, dataValidita, nomeFile, tipo (pdf|excel|altro),
note?.
 - I listini sono solo metadati: il file effettivo viene gestito altrove (cartella condivisa o sezione docu-
menti separata).
-

20. Produzione (/produzione , direzione-only)

Tre sotto-router: bom (distinte base), fasi , nc (non conformità).

20.1 Distinte base (BOM)

- Campi: `commessaId`, `aperturaId`, `stato`, `componenti[]`, `noteValidazione?`, `validataDa?`, `dataValidazione?`.
- Componenti: `tipo` (`profilo|vetro|ferramenta|guarnizione|accessorio`), `descrizione`, `codiceArticolo?`, `fornitoreId?`, `quantita`, `unitaMisura`, `lotto?`, `note?`.
- Stati BOM: `bozza`, `validata`, `in_produzione`, `completata`.
- Validazione consentita solo da `bozza` → `validata`.

20.2 Fasi produzione

- Campi: `commessaId`, `aperturaId?`, `distintaBaseId`, `fase`, `ordine`, `stato`, `operatore?`, `dataInizio?`, `dataFine?`, `checklistItems[]`, `note?`.
- Stati fase: `da_fare`, `in_corso`, `completata`, `non_conforme`.
- Checklist item: `descrizione`, `obbligatorio`, `completato`, `esito?` (`ok|non_conforme`), `note?`. Toggle dedicato `toggleChecklist`.
- `dataInizio` impostata su `in_corso`; `dataFine` su `completata`.

20.3 Non conformità (NC)

- Campi: `commessaId`, `aperturaId?`, `faseProduzioneId?`, `tipo`, `gravita`, `descrizione`, `azioneCorrettiva?`, `stato`, `segnalataDa`, `dataApertura`, `dataChiusura?`.
- Tipi: `materiale_difettoso`, `errore_taglio`, `errore_assemblaggio`, `vetro_rotto`, `ferramenta_errata`, `altro`.
- Gravità: `lieve`, `media`, `grave`, `bloccante`.
- Stati: `aperta`, `in_gestione`, `risolta`, `chiusa`. La chiusura imposta `dataChiusura`.

21. Preventivatori (/preventivatori)

Pagina hub con cards per azienda. L'accesso è aperto a tutti gli utenti autenticati.

21.1 Fivizzanese — Persiane (/preventivatori/fivizzanese/persiane)

Calcolo step-by-step:

1. **Selezione modello** (catalogo `shared/listini/fivizzanese.ts`).
2. **Quantità persiane standard** (con prezzo base modello).
3. **Quantità persiane con centinatura** (prezzo extra).
4. **Posa** (preset).
5. **Smontaggio vecchie** (opzionale, prezzo per unità).
6. **Promo** quando applicabile:
 - **Promo "RAL 6005 Opaco"**: prezzo aggiornato a **210 €** (precedentemente 200 €).
 - Quando la promo è attiva, viene aggiunto un **ricarico del 50%** sul totale persiane.
7. **IVA** (10% o 22%) — selettore.
8. Output:

- Riepilogo a video con righe: totale persiane, supplementi, centinatura, ricarica promo, smontaggio, imponibile, IVA, totale lordo.
- Esportazione **PDF** con jspdf-autotable; salvataggio automatico come documento preventivo sulla commessa (keepNome=true; nome file "Preventivo {cliente} - Fivizzanese.pdf").

21.2 Punto del Serramento — Persiane (/preventivatori/punto-del-serramento/persiane)

Stesso schema più:

- **Sconto** percentuale modificabile, **max 30%** (clamp server-side e client-side).
- **IVA** (10% o 22%) — selettore.
- Output PDF analogo con righe **lordo, sconto, imponibile, IVA, totale**.

22. Archivio (/archivio)

22.1 Soft-archive

- Endpoint `commesse.archive(id)` : imposta `archivedAt = ISO now`.
- Endpoint `commesse.restore(id)` : imposta `archivedAt = null`.
- Lo stato (`stato`) e i collegamenti (file, aperture, interventi, prodotti) **non vengono toccati**. Restore = la commessa torna esattamente com'era.

22.2 Visibilità

- Le commesse soft-archivate sono escluse da:
 - `commesse.list` (scope default `exclude`).
 - `commesse.stats` e `commesse.byPriorita`.
 - Board Kanban.
 - Dashboard.
 - Notifiche proattive (vedi §27).
 - Classifica venditori.
- Restano accessibili a `/archivio` (scope `only`) e tramite link diretto `/commesse/:id`.

22.3 Pagina Archivio

- Lista con search per codice/cliente/città/indirizzo.
- Per ogni riga: codice, stato originale (badge), priorità, cliente, indirizzo, data apertura, data archiviazione, pulsanti **Ripristina** e **Apri scheda**.
- Conferma esplicita prima del ripristino.

22.4 Pulsanti nella scheda commessa

- **Archivia** (se non archiviata) — conferma non-destructive.
- **Ripristina** (se archiviata).
- Banner di stato "Commessa archiviata" in alto quando applicabile.

23. Classifica venditori (/classifica)

23.1 Regola di conteggio

Per ogni utente con ruolo `commerciale` e `attivo === true`, conta le commesse che soddisfano TUTTE queste condizioni:

1. `assegnatoA === utente.id`.
2. `archivedAt === null` (no soft-archive).
3. `stato` $\in$ `{misure_esecutive, aggiornamento_contratto, fatture_pagamento, da_ordinare, produzione, ordini_ultimazione, attesa_posa, finiture_saldo, interventi_regolazioni}` — esclusi `preventivo` (non ancora "vera" commessa) e `archiviata` (chiusura).

23.2 Ordinamento e ranking

- Ordine: count desc, tie-break alfabetico (`cognome nome`).
- Rank **competition (1224)**: due pari-merito al primo posto → il successivo è 3°.

23.3 Podio "intelligente"

- Il podio mostra fino a **3 score-tier** (gradini), non 3 persone:
 - Tutti i pari-merito condividono lo stesso gradino.
 - Se due venditori sono primi a pari merito, il gradino di argento può essere vuoto e il successivo è terzo con rank 3.
- Se **tutti** hanno lo stesso punteggio → un solo gradino oro con banner *"PARI MERITO TOTALE — SIE-TE TUTTI PRIMI"*.
- Layout visuale: 2 — 1 — 3 (vincitore al centro), gradino tallone più alto in oro.

23.4 Interattività e "battute"

- **Coriandoli CSS** all'apertura della classifica, replay su click.
- **Banner Commento della giuria** (amber, prominente, rotazione automatica ogni 5 s + click).
- **Bolla citazione vincitore** (rotazione click).
- **Click su qualunque atleta** → coriandoli + toast spiritoso ("X scalda i motori!", "Occhio, X è in modalità squalo", ...).
- **Count-up** animato da 0 al valore finale per ogni numero commesse.
- **Tag "Cucchiaino di legno"** all'ultimo, badge "-N dal 1°" per gli inseguitori, fun-fact rotante nel footer.
- Zero dipendenze esterne per gli effetti (CSS keyframes + sonner già presente).

23.5 Empty state

- Nessun utente `commerciale` attivo → "Podio vuoto, eco assordante. Aggiungi utenti con ruolo «commerciale» dalla pagina Utenti".
-

24. Utenti (/utenti , direzione-only)

24.1 Funzionalità

- Lista utenti con filtro per ruolo + search testuale.
- Creazione, modifica, eliminazione (tutte `adminProcedure`).
- Multi-ruolo (1–3 ruoli per utente).
- Toggle `attivo/inattivo` .
- Password gestita solo lato server (hashing scrypt, vedi §3.3). Il campo `hasPassword` (bool) è restituito al client al posto del contenuto.

24.2 Gating

- Rotta `/utenti` protetta da `<RequireDirezioe>` .
- Sidebar mostra la voce **Utenti** solo a utenti `direzione` .

24.3 Email

- Univocità email enforced lato server (`utenti.create` rifiuta duplicati case-insensitive).

25. Notifiche proattive (/ dropdown header)

25.1 Soglie per priorità (giorni dall'ultimo `updatedAt`)

Priorità	Soglia
bassa	7 gg
media	5 gg
alta	3 gg
urgente	1 gg

Una volta superata la soglia, la notifica viene rigenerata **ogni giorno** finché non c'è una nuova `update` sulla commessa.

25.2 Daily reminder su stati bottleneck

Per i seguenti stati ogni giorno (a partire dal secondo) viene generata una notifica per l'owner/creator:

- `aggiornamento_contratto`
- `fatture_pagamento`
- `da_ordinare`

Severità: `urgent` se età ≥ 5 gg, `warning` se ≥ 3 gg, altrimenti `info` .

25.3 Stato → ruolo (routing per ruolo)

Stato	Ruolo destinatario
da_ordinare	ordini
misure_esecutive	tecnico_rilievi
fatture_pagamento	amministrazione
finiture_saldo	amministrazione

I destinatari per ruolo ricevono una notifica `stato_role` aggiuntiva mentre la commessa è in quello stato.

25.4 Esclusioni

Le notifiche **NON** vengono generate per:

- Commesse con `stato === "archiviata"` (chiusura del flusso).
- Commesse soft-archivate (`archivedAt !== null`).

25.5 Endpoint

- `notifiche.list` → array notifiche per utente corrente.
- `notifiche.count` → numero notifiche. Entrambi richiedono utente autenticato; ritornano array vuoto/0 quando `ctx.user` è assente.

26. Dashboard (/)

26.1 KPI principali

- Cards Commesse, Anomalie, Ticket, Garanzie con conteggi e split per stato dove rilevante.
- Polling live (interval) tramite `live0pts`.

26.2 Settimana interventi

- Slot 7 giorni con conteggi di interventi pianificati per giorno + tipo.

26.3 Commesse recenti / per priorità

- Lista delle commesse aperte ordinate per priorità (Urgente in alto), con quick link alla scheda.
-

27. Integrazioni esterne (/integrazioni)

27.1 Microsoft To Do

- Sincronizzazione dei task operativi generati dalle commesse (su roadmap operativa, non ancora full-featured).

27.2 Google Calendar

- Gli interventi (Rilievo, Posa, Assistenza) **devono** essere proiettabili sul calendario aziendale con i dati: data/ora, titolo (tipo + cliente), indirizzo lavoro, telefono.

27.3 Notifiche owner

- Endpoint `system.notifyOwner` invia notifiche al servizio Manus Notification Service quando configurato (env `BUILT_IN_FORGE_API_URL` + `BUILT_IN_FORGE_API_KEY`). Richiede ruolo `admin`.

27.4 Google Maps

- Componente `<Map/>` lato client carica lo script via proxy interno (`VITE_FRONTEND_FORGE_API_URL`) con API key dedicata frontend (non sensibile, key di proxy).

28. Persistenza

28.1 KV store

- Tabella `kv_store(key text primary key, data jsonb, updated_at timestampz)`.
- Ogni router business possiede una o più "raccolte" persistenti: `clienti`, `commesse`, `aperture`, `interventi`, `anomalie`, `tickets`, `ticket_allegati`, `squadre`, `garanzie`, `verbal`, `fornitori`, `fornitori_ordini`, `fornitori_listini`, `produzione_distinte`, `produzione_fasi`, `produzione_nc`, `preventivi_documenti`, `reclami`, `rifacimenti`, `timeline_steps`, `utenti`.

28.2 Lifecycle

- **Bootstrap.** All'avvio `bootstrapAll()` legge ogni raccolta. Tre stati possibili per la singola key:
 - `firstBoot = true`: nessuna row presente → callback `onLoad` può popolare seed.
 - `firstBoot = false, loaded = true`: row presente, dati ripristinati con `reviver Date`.
 - `firstBoot = false, loaded = false`: errore transiente; **i save sono bloccati** finché la background recovery non riesce a leggere lo stato dal DB.
- **Save.** Debounciato 200 ms; usa `sql.json()` per evitare doppia encoding. Retry exponential backoff su errori transienti (EAI_AGAIN, ENOTFOUND, ECONNREFUSED, ECONNRESET, ETIMEDOUT, ENETUNREACH, EHOSTUNREACH, EPIPE).
- **Shutdown.** Handler `SIGTERM/SIGINT` chiamano `flushAll()` per non perdere mutazioni in flight, poi chiudono la pool.

28.3 Recovery

- Se ensureSchema o load falliscono dopo i retry → `backgroundRecover` (max 30 tentativi a intervalli di 5 s) finché ogni raccolta non risulta `loaded = true`. Nessun seed viene applicato in caso di fallimento (per non sovrascrivere dati reali con un seed accidentale).

28.4 Legacy double-encoded payload

- Riconosciuto a load: se `data` è stringa JSON, viene parsata e schedulata una riscrittura corretta.

28.5 Vincoli su Postgres

- Connessione Postgres-js: max 5 connessioni, `idle_timeout=20s`, `connect_timeout=30s` (Railway internal DNS warmup).
 - SSL automatico quando l'URL contiene `sslmode=require` o l'host Railway.
-

29. UI/UX e componenti

29.1 Sistema

- Tailwind + shadcn/ui (Button, Card, Badge, Dialog, AlertDialog, Avatar, Input, Label, Select, Switch, Tabs, Tooltip, DropdownMenu).
- Icone lucide-react.
- Toast: sonner.
- Componente `<SearchSelect>` riutilizzato per dropdown ricercabili (utenti, squadre, fornitori, ecc.).
- Componente `<ConfirmDialog>` riutilizzato per: cancellazioni, archiviazioni, "Procedi comunque" del doc gate, ripristino archivio.
- Componente `<FilePreviewDialog>` per preview PDF/immagini.

29.2 Sidebar

- Voci dinamiche; quelle con `direzioneOnly` filtrate per ruolo.
- Resizable (larghezza personalizzabile da utente).
- Avatar utente in basso con menù: nome, email, Esci.

29.3 Mobile

- Layout responsivo. Sidebar collassata in modalità mobile; nasconde scritte tenendo solo icone.

29.4 Empty states

- Tutte le pagine principali hanno empty state esplicito con istruzioni sul prossimo passo.
-

30. Errori e telemetria

30.1 Convenzioni errori

- Gli errori non-tRPC sono ritornati come `Error` con messaggio human-readable in italiano.
- Errori "marker" usano un **prefisso** come `DOC_GATE_BLOCKED:` che il client identifica per offrire UI dedicata.

30.2 Logging

- Tutto il logging della persistenza passa da `console.log/warn/error` con prefisso `[persistence]`.
- I save e i load mostrano sempre il conteggio degli elementi per raccolta.

31. Roadmap aperta (lavori noti)

31.1 Sicurezza

- **Rotazione credenziali** committate nello storico Git + **purge history** (BFG/ `git filter-repo`).
- CSP (Content Security Policy) tarata su Vite, blob preview, Maps proxy.

31.2 Ottimizzazione

- **Per-file storage** dei documenti: oggi ogni upload riscrive tutta la JSONB della raccolta `preventivi_documenti`. Migrazione consigliata verso una row kv per documento o, meglio, una tabella `documents` con colonna `bytea` /object storage esterno. Richiede backup DB prima di toccare i dati.
- Aggregato dashboard in un unico endpoint per ridurre il fan-out lato client.

31.3 UX

- Drag&drop diretto sulle colonne del Kanban (oggi solo bottoni avanza/indietro).
- Confetti hardware-accelerati (opzionale).

31.4 Integrazioni

- Esportazione CSV/Excel commesse, clienti, anomalie.
- Push notifications mobile.

32. Glossario

- **Commessa.** Progetto specifico di vendita+installazione per un cliente.
- **Apertura.** Singolo serramento (finestra, porta, ...) all'interno di una commessa.
- **Stato.** Posizione corrente della commessa nella state machine.
- **Soft-archive.** Stato secondario, ortogonale allo stato del workflow: la commessa è nascosta dalle viste operative ma tutto è preservato.

- **Doc gate.** Vincolo per cui un avanzamento di stato richiede l'upload di documenti previsti per lo stato corrente.
 - **Bypass.** Conferma esplicita dell'operatore tramite `force: true` per superare il doc gate.
 - **Tier (classifica).** Un gruppo di venditori con lo stesso conteggio commesse; condividono il rank.
 - **Pari merito.** Più persone con lo stesso conteggio. Condividono il rank e il gradino del podio.
 - **Indirizzo di residenza.** Indirizzo fiscale del cliente (uso amministrativo).
 - **Indirizzo di lavoro.** Indirizzo del cantiere (uso operativo per commesse, calendario, mappe).
 - **Tipo detrazione.** Modalità fiscale richiesta dal cliente: `ecobonus` o `ristrutturazione`.
-

33. Cronologia significativa

- v3.0 — Riallineamento completo del PRD al codice corrente: dual address, tipoDetrazione, dataConsegnaIndicativa, soft-archive, Archivio, Classifica venditori, doc-gate con bypass, hardening sicurezza (scrypt, JWT fail-hard, mimeType allowlist, rate-limit login, security headers, session eviction, logout server-side), assegnazione utente modificabile, preventivatori Fivizzanese e Punto del Serramento, Planning con joined info.
- v2.x — Versione precedente (PDF allegato in repo come riferimento storico).
- v1 — Documento originale.